BUILT AND TESTED

Save AI results to the database

A repeat run now loads in 0.4 seconds with 0 tokens - instead of two to three minutes and about 33,000 tokens

XenAnalyticTransl - PLC Logic Builder AI layer

Siti Syaziyah | 22 October 2026

Before: every click re-translated every scenario

Even when nothing had changed since the last run

7

AI calls per click

One per scenario - all seven, every time

~33K

tokens per full run

About 4,700 tokens per scenario

~5

full runs per day

Groq free tier: ~200K tokens a day

Most of those calls repeated work already done. On a free tier, that limit decided how much we could test.

Translate once, save it, reuse it

The database remembers every result; the AI is called only for what is new

7 AI calls

First launch

Database is empty. Translate all seven and save them.

0 AI calls

Click again, nothing changed

All seven load straight from the database.

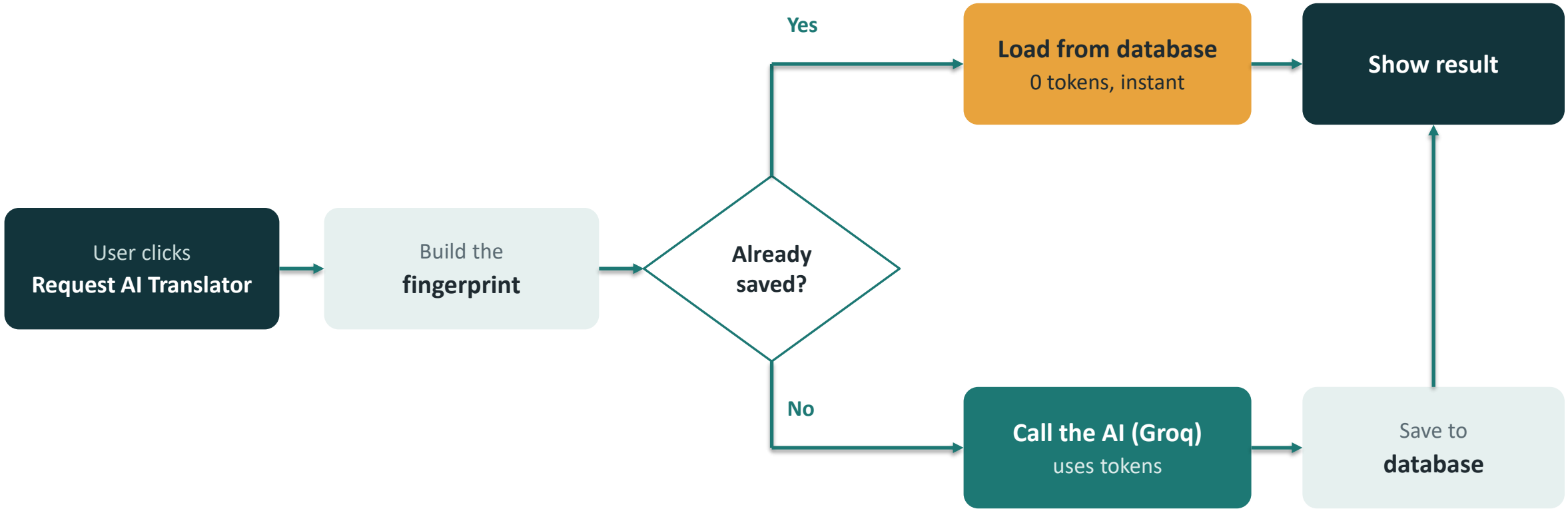
1 AI calls

One scenario edited

Translate that one. Load the other six from the database.

Measured: a repeat run of all 7 Motor scenarios loaded in 0.4 s with 0 tokens. Editing one scenario costs about 4,700 tokens instead of 33,000.

How it works, for each scenario



Repeats for all 7 scenarios, then shows a short summary.

A failed AI call is never saved. It shows as an error, so it is retried next time.

The fingerprint decides: reuse or translate

Same inputs give the same code. Change one letter and the code changes

1. Scenario text
2. Scenarios it refers to
3. Property list
4. Model and output type
5. Translation rule version



If anyone edits the translation rules, input 5 changes and old results are translated again instead of reused.

The catch it handles

Operation refers to Auto Run Ready. Edit Auto Run Ready and Operation's answer must change too - even though its own text did not.

Because input 2 is included, both are refreshed.

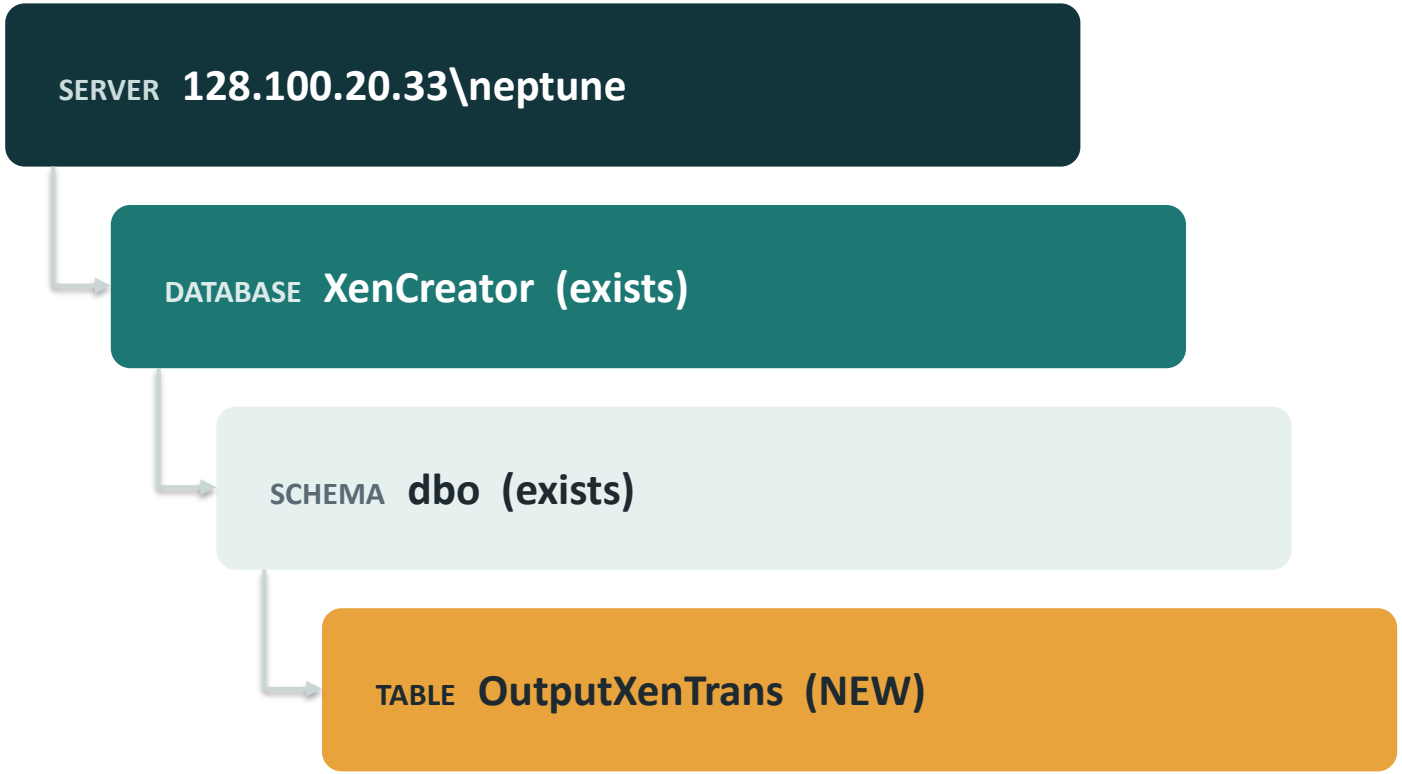
Change made

Requests used to carry all 7 scenarios, so one edit changed all 7 fingerprints. Now only the scenarios actually mentioned are sent.

Bonus: smaller requests on every call.

Where it is stored

A new table inside the existing XenCreator database - created and in use



Full name

`XenCreator.dbo.OutputXenTrans`

"OutputXenTrans.dbo" would read as a new database.

History is kept

Each run adds a row instead of overwriting one.

The app always loads the newest matching row.

Old rows stay, so runs and models can be compared later.

What one saved row holds

Column	Holds	Example
CacheKey	The fingerprint - used to find a match	`7f3a9c21...c91e`
Equipment, ScenarioName	Which scenario this is	`Motor, Trip`
ScenarioText	The scenario exactly as translated	`When Trip Signal is On...`
OutputTarget	Which output was asked for	`Pseudocode`
Provider, Model	Which AI produced it	`Groq, qwen3.8-27b`
Code	The generated pseudocode	`IF Trip Signal == ON THEN...`
Explanation, Warnings	What the AI explained and flagged	`["Trip Alarm never OFF"]`
VariablesUsed	Names the AI used - SCL results are re-checked on load	`["Trip Signal"]`
PromptTokens, CompletionTokens	Tokens used - to measure the saving	`4,422  /  296`
CreatedAt	When it was saved	`2026-10-21 15:47`

VariablesUsed was added during the build: without it a reloaded SCL result could not keep its tick or warning mark. The script is in the appendix.

What changed in the code

Three new files, four changes - the AI call and the rules stayed as they were

File	Change	Type
AI / ResultStore.cs	Connects, checks the table, loads and saves results, builds the fingerprint	New
Reference / sql / OutputXenTrans.sql	The table script - safe to run twice	New
Reference / sql / Setup-XenTransDb.ps1	One-command setup; asks for the password hidden	New
Studio / MainWindow.xaml.cs	Checks the database before the AI; sends only referenced scenarios	Changed
Studio / MainWindow.xaml	"Ignore saved" checkbox	Changed
AI / Models.cs	A result can be marked as loaded from the database	Changed
AI / XenAnalyticTransl.AI.csproj	SQL Server package, plus a patched security component	Changed

Unchanged: LlmTranslator.cs (the AI call) and PromptBuilder.cs (the translation rules).

What the user sees

Status bar after a repeat run

```
Done: 7 scenario(s) in 0.4s  |  0 translated (0 tokens), 7 loaded from DB  |  7 clean
```

Result header

AI Translation Result (Pseudocode)

Loaded from DB successfully

0.1 s qwen3.8-27b

loaded from DB, saved 21 Oct 15:47

0 tokens used now

Rows in XenCreator.dbo.OutputXenTrans

Id	Scenario	Output	CreatedAt
1	Operation	Pseudocode	21 Oct 15:47
7	Trip	Pseudocode	21 Oct 15:47
8	Trip	Pseudocode	22 Oct 10:47

Row 8 is Trip re-translated after an edit. Row 7 stays as history.

Results stay on screen while the app is open. Click between scenarios, or switch Motor to Valve and back - each keeps its output.

Tested before use

21 automated checks on a throwaway local database, then on the real server

Check	Result
The table script runs - and runs a second time without errors	**Pass**
A saved result loads back identically, line breaks and warnings included	**Pass**
Failed results are never saved	**Pass**
History is kept, and the newest saved row wins	**Pass**
Editing Trip re-translates only Trip	**Pass**
Editing Auto Run Ready also refreshes Operation and System Handle	**Pass**
Unreachable server: a plain message, and the password never appears	**Pass**
Live: a Groq answer saved, then served from the database with 0 tokens	**Pass**
Real server: 7 Motor scenarios loaded from DB in 0.4 seconds	**Pass**

Four decisions, now settled

1

The login stays out of the code

Stored in the XENTRANS_DB setting, like the API keys. Windows sign-in is refused on this server, so it uses the rndsa login; the setup script asks for it hidden.

2

The table is created

XenCreator.dbo.OutputXenTrans was created with OutputXenTrans.sql and is in use.

3

Database down? The app still works

It checks once per run. If the server cannot be reached, it carries on with the AI and says why in the status bar.

4

Saved results are frozen - handled

"Ignore saved" forces a fresh translation. The new result is saved too, and is the one loaded next time.

What is next

1. **Measure the saving over time**
   Tokens per day, read from the table

2. **Name the six New Instance tags**
   The biggest single limit on SCL quality

3. **Compile the output in TIA Portal**
   Not done yet - the real test of correctness

4. **Tidy the scenario wording**
   Valve names missing from the data

THE RESULT

Repeat runs now cost nothing:
0.4 seconds and 0 tokens, instead of two to three minutes and about 33,000 tokens.

The free tier now stretches across far more testing.

Appendix: the table script, as run

```
USE XenCreator;
GO
IF OBJECT_ID(N'dbo.OutputXenTrans', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.OutputXenTrans (
        Id               INT IDENTITY(1,1) PRIMARY KEY,
        CacheKey         CHAR(64)       NOT NULL,
        Equipment        NVARCHAR(50)   NOT NULL,
        ScenarioName     NVARCHAR(200)  NOT NULL,
        ScenarioText     NVARCHAR(MAX)  NOT NULL,
        OutputTarget     NVARCHAR(30)   NOT NULL,
        Provider         NVARCHAR(100)  NOT NULL,
        Model            NVARCHAR(100)  NOT NULL,
        Code             NVARCHAR(MAX)  NOT NULL,
        Explanation      NVARCHAR(MAX)  NULL,
        VariablesUsed    NVARCHAR(MAX)  NULL,
        Warnings         NVARCHAR(MAX)  NULL,
        PromptTokens     INT            NULL,
        CompletionTokens INT            NULL,
        ElapsedMs        INT            NULL,
        CreatedAt        DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME()
    );
    CREATE INDEX IX_OutputXenTrans_CacheKey
        ON dbo.OutputXenTrans (CacheKey, CreatedAt DESC);
END
GO
```